Advance Python Programming

National Vocational and Technical Training Commission (NAVTTC)

Assignment 1	
Dr. Rabia Tehseen & Fraz Aslam (University of Central Punjab, Lahore)	
Name:	**Marks**: / 300
Deadline: 13-March-2026, Friday	

Instructions

- **Section A:** Answer **all 10 short questions**.
- Each answer should be in **5–10 lines** with **proper explanation**.
- Do not write only definitions. Use **logic, reasoning, and examples** where needed.
- **Section B:** Attempt **both long coding questions**.
- Write **clean**, **modular** Python **code** with meaningful variable names and proper structure.
- Write in your own words, **no use of Generative AI**, unless for understanding.
- Bring a hardcopy **printout** of your assignment on the day of submission. This **title page is MANDATORY**.

Section A — Short Conceptual Questions

Q1. String Immutability and Efficiency

Python strings are immutable. Explain why this matters in real programs. Also discuss why repeatedly using += inside a loop to build a large string may be a poor design choice, and what better alternative can be used.

Q2. Short-Circuit Logic in Validation

Consider this condition:

```
if text and text.strip().isdigit() and int(text) > 10:
```

Explain how Python evaluates this condition step by step. Why is the order of checks important here, and what possible bug could happen if the order is changed?

Q3. for-else / while-else Logic

Many students think else is only for if. Explain the purpose of else with loops in Python. Give one logical situation where loop-else is cleaner than using an extra flag variable.

Q4. Lists, Copying, and Hidden Bugs

Why can this code create unexpected behavior?

```
a = [[1, 2], [3, 4]]
b = a.copy()
b[0][0] = 99
```

Explain the issue clearly. Also state how this differs from copying a simple one-dimensional list.

Q5. Set vs List for Membership

Suppose you need to check whether a username has already appeared in 50,000 records. Explain why a set is usually a better choice than a list for this task. Also mention one situation where a set would be the wrong choice.

Q6. Dictionaries as Structured Records

A student record can be stored as a dictionary like:

```
{"name": "Ali", "marks": [78, 82, 90], "city": "Lahore"}
```

Explain why a dictionary is often better than a tuple for this kind of data. Then mention one case where a tuple may still be the better choice.

Q7. Stable Sorting and Logical Ordering

Python sorting is stable. Explain what this means. Then explain how stable sorting allows you to sort student records first by name and then by marks without losing the earlier ordering logic.

Q8. Comprehensions vs Normal Loops

A comprehension can often replace a loop in one line. Explain when using a comprehension is a good idea, and when it becomes a bad idea even if it still works correctly. Your answer should focus on readability and maintainability, not only syntax.

Q9. Parsing: split() vs Regex Mindset

Suppose you are given text like:

"name=Ali, age=20, city=Lahore"

Explain how this can be parsed using simple string methods such as split() and strip(). Then explain when regex becomes useful, and when using regex would be unnecessary overengineering.

Q10. VS Code Interpreter and Environment Confusion

A student installs a package using pip, but the code in VS Code still shows ModuleNotFoundError. Explain two or three logical reasons why this can happen. Your answer should mention interpreter selection, virtual environments, and workspace/project setup.

Section B — Long Coding Questions

Q11. Project 1: Student Record Analyzer and Report Generator

Problem Statement

You are required to develop a **Student Record Analyzer** for a teacher. The program will read raw student data, clean it, validate it, organize it, analyze it, and print a formatted report.

Input Format

The data will be given as a list of raw strings like this:

```
records = [
    "101, Ali, CS, 78|82|91, city=Lahore",
    "102, Sara, SE, 88|79|85, city=Karachi",
```

```
    "103, Ahmed, CS, 90|92|89, city=Islamabad",
    "101, Ali, CS, 78|82|91, city=Lahore",
    "104, Hina, EE, 65|70|72, city= Lahore ",
    "105, , CS, 80|81|82, city=Multan",
    "106, Zoya, SE, 91|x|87, city=Karachi"
]
```

Requirements

Build a Python program that does all of the following:

1. **Clean the input**
    - Remove unnecessary spaces.
    - Handle missing or malformed fields.
    - Ignore or report invalid records.
2. **Parse each line**
    - Extract:
        - student ID
        - name
        - department
        - marks list
        - city from key=value format
3. **Validate data**
    - Student ID must be numeric.
    - Name should contain only alphabetic characters and spaces.
    - Marks must all be valid integers.
    - Empty names or invalid marks should be treated as invalid records.
4. **Use appropriate data structures**
    - Use a **dictionary** for each student record.
    - Use a **set** to detect duplicate student IDs.
    - Use **lists** for marks.
    - Use **nested dictionaries** or grouped dictionaries for department-wise summaries.
5. **Perform analysis**
    - Compute total, average, and grade for each valid student.
    - Count how many students belong to each department.
    - Find the top-performing student(s).
    - Find department-wise average marks.
    - Count frequency of cities.

6. **Sorting**
    - Sort students by:
        - department
        - descending average marks
        - name
7. **Formatted output**
    - Print a clean report using **f-strings** with alignment/padding.
    - Example: columns for ID, Name, Dept, Average, Grade, City.
8. **Comprehensions**
    - Use at least:
        - one list comprehension
        - one dictionary comprehension
        - one set comprehension
9. **Modular design**
    - Divide your program into multiple functions.
    - Create a separate file utils.py for helper functions.
    - Use if __name__ == "__main__": properly in main.py.
10. **Error handling and logic**
    - Use guard clauses where appropriate.
    - Avoid unnecessary nested loops.
    - Clearly comment any tricky logic.

Expected Concepts Covered

Strings, parsing, validation, conditions, loops, lists, tuples, sets, dictionaries, comprehensions, sorting with key, modules, formatted output, workflow with multi-file project structure.

Additional Challenge

Add a feature that allows the user to search by city or department and display matching student records only.

Q12. Project 2: Mini Command-Line Log Analyzer

Problem Statement

Develop a **Mini Log Analyzer** for a small software system. The system stores logs as text lines. Your program must read, clean, parse, classify, and analyze these logs to produce useful summaries.

Sample Input

```
logs = [
    "2026-03-01 09:15:22 | user=ali | action=login | status=success",
    "2026-03-01 09:17:10 | user=sara | action=upload | file=report.pdf | status=success",
    "2026-03-01 09:18:02 | user=ali | action=logout | status=success",
    "2026-03-01 09:20:11 | user=ahmed | action=login | status=failed",
    "2026-03-01 09:21:45 | user=ahmed | action=login | status=failed",
    "2026-03-01 09:22:50 | user=ahmed | action=login | status=success",
    "2026-03-01 09:25:00 | user=sara | action=delete | file=old.docx | status=success",
    "INVALID LINE HERE",
    "2026-03-01 09:30:00 | user=ali | action=upload | file=image.png | status=success"
]
```

Requirements

Write a Python program that performs the following tasks:

1. **Clean and validate logs**
    - Ignore or separately store malformed lines.
    - Strip unwanted spaces and clean fields.
2. **Parse log entries**
    - Extract date/time, user, action, file (if present), and status.
    - Use string methods primarily.
    - You may optionally use light regex if justified.
3. **Store data properly**
    - Use a list of dictionaries for parsed logs.
    - Use nested dictionaries where useful.
    - Use sets to keep track of unique users, actions, and file types.
4. **Generate summaries**
    - Total valid logs
    - Total invalid logs

- Number of unique users
- Frequency of each action
- Frequency of each status
- Most active user
- Most common file extension
- Number of failed login attempts per user

5. **Logical analysis**

    - Detect suspicious behavior:
        - more than 2 failed logins by the same user
        - delete action after repeated failures
        - same user performing too many actions in a short sequence

6. **Searching and filtering**

    - Allow the user to:
        - search logs by username
        - filter by action
        - filter by status
        - print only failed events

7. **Formatted reporting**

    - Display logs and summaries in aligned columns using f-strings.
    - Show a final analytical report section.

8. **Use Python features meaningfully**

    - At least one use of:
        - enumerate()
        - zip()
        - list/dict/set comprehension
        - sorting with key
        - safe dictionary access using .get()

9. **Project structure**

    - Create:
        - main.py
        - parser.py
        - analyzer.py
    - Use imports correctly.
    - Use if __name__ == "__main__":.

10. **Extension**

    - Save the final report to a text file.

- Optionally use datetime to calculate time gaps between events.

Expected Concepts Covered

Advanced strings, validation logic, parsing patterns, loops, dictionaries, sets, lists, comprehensions, sorting, modular programming, standard library usage, VS Code project workflow.

Additional Challenge

Add a menu-driven interface so the user can repeatedly choose options such as:

- View all logs
- Search by user
- Show suspicious activity
- Export report